

ISE 331: Fundamentals of Computer Security

Practice Problems for Lectures 24 to 26 (Software Security)

Directions: Below is a set of exercises and problems you should do for practice. Answering this set of questions should be just one part of your preparation for quizzes and exams. In addition to solving these problems you should:

- Review the lecture notes.
- Review your homework assignments.
- Form a study group and test each other on the material.

Note: The solution is provided on the last page. It is strongly recommended that you work on the questions first before going to the solution.

Part I. Multiple Choice

1. A Time-of-Check to Time-of-Use (TOCTOU) vulnerability occurs primarily because the state of a resource can be modified between the moment it is verified and the moment it is acted upon. Which of the following best describes the underlying cause?
 - A. The use of symbolic links within the filesystem structure.
 - B. The lack of atomicity between the check and the use operations.
 - C. The operating system's inability to handle multiple file descriptors.
 - D. The high execution speed of modern multi-core processors.
2. In the context of race conditions, a “critical section” in a program is best defined as a block of code that:
 - A. Runs with the highest possible CPU priority to ensure fast execution.
 - B. Only contains calls to the Linux kernel's system call interface.
 - C. Can only be executed by one thread or process at the same time.
 - D. Is guaranteed to be reentrant across all possible signal handlers.
3. In a multi-threaded environment, what is a primary security risk of resource sharing between threads?
 - A. One thread can accidentally overwrite the registers of another thread.
 - B. Threads can pop and push each other's stack, resulting in data clobbering.
 - C. A thread can modify a global variable after another thread has checked its value.
 - D. The operating system cannot distinguish different threads for the purpose of scheduling.
4. What is the primary security benefit of Extended Page Tables (EPT) in a virtualized environment?
 - A. It allows a guest operating system to have direct, unmonitored access to the host's physical RAM.
 - B. It provides address translation that isolates virtual machines from each other and the host's physical hardware.
 - C. It replaces the standard Page Table to make available memory space larger, avoiding buffer overload.
 - D. It prevents userspace applications from performing system calls that are above their privilege level.
5. In a computer with virtual machine installed, both the host kernel and the virtual machine kernel need to run with the highest privilege (Ring 0). A modern virtual machine solves this issue by:
 - A. Host kernel runs at Ring -1 and the virtual machine kernel runs at Ring 0.
 - B. Host kernel runs at Ring 0 and the virtual machine kernel runs at Ring -1.
 - C. Host kernel runs at Ring 0 and the virtual machine kernel runs at Ring 1.
 - D. Host kernel runs at Ring 1 and the virtual machine kernel runs at Ring 0.

6. Which is a typical operation procedure for an operating system kernel?
 - A. Read from userspace, execute instructions, write to userspace
 - B. Write to userspace, execute instructions, read from userspace
 - C. Read from userspace, execute instructions, write to userspace, return to userspace
 - D. Write to userspace, execute instructions, read from userspace, return to userspace
7. A software security researcher is choosing between static and dynamic analysis to find vulnerabilities in a codebase. Which of the following is a primary disadvantage of using only dynamic analysis?
 - A. It generates false positives by reporting bugs that cannot actually happen.
 - B. It requires the original source code to perform proper analysis.
 - C. It can only detect vulnerabilities that are triggered during testing.
 - D. It cannot provide information about the state of memory at the time of a crash.
8. An information flow-sensitive taint analysis tool is responsible for:
 - A. Distinguish tainted/untainted variables with multiple assignments.
 - B. Distinguish tainted/untainted statements within conditionals (if-else).
 - C. Distinguish tainted/untainted function calls with different contexts.
 - D. Distinguish tainted/untainted statements in implicit flows.
9. Which sensitivity allows an analysis to distinguish between different calls to the same function, preventing taint from one function call from leaking into another function call?
 - A. Flow sensitivity
 - B. Context sensitivity
 - C. Path sensitivity
 - D. Pointer sensitivity
10. One statement in the sourcecode leads to tainting a function that must remain untainted. However, this statement is preceded by a condition `if (pass)`, and the value of `pass` is always `false`. If a taint analysis tool ignores this logic and reports tainted, it is missing:
 - A. Flow sensitivity
 - B. Context sensitivity
 - C. Path sensitivity
 - D. Pointer sensitivity

Part II. True or False

1. In a multi-threaded environment, threads within the same process share the same virtual memory space and registers.
2. Standard library functions like `malloc()` and `free()` are unsafe to use inside signal handlers because they are inherently non-reentrant.
3. The `clone()` system call provides more granular control over what resources (like memory and file descriptors) are shared between a parent and child compared to the traditional `fork()`.
4. In a microkernel architecture, a vulnerability in a filesystem driver typically presents a higher risk of full system takeover compared to a monolithic kernel.
5. Kernel memory sits at the lower half of the virtual memory addresses.
6. In a computer with multiple virtual machines, there is only one Page Map Level 4 (PML4) managing entire virtual memory, and each virtual machine occupies part of it.
7. In the context of program analysis, a “sound” tool is one that never misses a real error (no false negatives).

8. Program analysis tools are usually sound or complete, but not both.
9. In the context of program analysis, a “complete” tool is one that only reports real bugs (no false positives).

Part III. Short Answer

1. How do "filesystem mazes" (using deeply nested directories or symbolic links) help an attacker exploit a race condition?
2. Describe how a signal/interrupt can lead to a race condition even in a single-threaded program.
3. Explain how the function below can have unexpected results in a multi-threaded environment. Provide a simple solution.

```
unsigned int num = 0;
void *thread_main(int arg) {
    while (1) {
        num++;
        num--;
    }
}
```

4. In a system using a monolithic kernel architecture, what is the primary security implication of including device drivers within the main kernel binary?
5. To transfer data between kernel and userspace, it is necessary to handle user data carefully and only access it with `copy_to_user()` and `copy_from_user()`. What is the reason for that? List two issues that may arise if we don't follow the practice above.
6. Perform taint analysis for the source code below.

```
int printf(untainted char *s);
tainted char *get_user_input();

void process(int code) {
    char *msg;
    char *input = get_user_input();

    if (code > 10) {
        msg = input;
    } else {
        msg = "Access Denied";
    }

    if (code < 5) {
        printf(msg);
    }
}

// Main call
process(0);
```

7. Perform taint analysis for the source code below.

```
int printf(untainted char *fmt);
tainted char *fgets();
```

```
char *wrapper(char *p) {
    return p;
}
```

```
// Call 1
char *a1 = fgets();
char *r1 = wrapper(a1);
printf(r1);
```

```
// Call 2
char *a2 = "Safe String";
char *r2 = wrapper(a2);
printf(r2);
```

8. A mutational fuzzer starts with a 'seed corpus'. Why is the quality of this initial collection of inputs critical for the fuzzer's success?

Part I Solution:

1-5 BCCBA 6-10 CCDBC

Part II Solution:

1-5 FTTF 6-9 FTFT

Part III Solution:

1. These mazes force the kernel to spend more time resolving the file path. By slowing down the kernel, the attacker increases the time between file opening and performing action, making it easier to swap the file during that window.
2. Signals can pause the execution of a program, including inside a critical section. If the signal handler modifies a global variable that the main program just checked but is yet to use, it becomes a race condition and can result in clobbered data.
3. Race conditions can happen to the function in multi-threaded environment. Instructions `num++` and `num--` are not atomic, they each become three steps after compiling into assembly code: reading the value from memory, modifying it in a register, and writing it back to memory. If multiple threads **interleave** these steps, they can clobber each other's result, leading to inconsistent values.
(Note: You can also write down the flow on slide 26 of Lecture 24, and explain from here)
Solution: Copy the value to a local variable before modification, or use mutexes so only one thread can execute the critical section at a time.
4. In a monolithic kernel, drivers are loaded into the same address space as the kernel and operate with the same high privilege levels (e.g., Ring 0). As a result, any vulnerability in a driver can lead to full system compromise.
5. We handle user data carefully to avoid corrupting kernel memory. Corrupting kernel memory can lead to system crashes, bricking the system (can't bootup), privilege escalation and interfering with other processes.

6. Names: input – i, msg – m, code – c, use the name p for parameter in process(), use the name ai and am for local variables input and msg in process().

process(0): p-1 >= untainted

input = get_user_input(): ai >= tainted

path sensitivity check: msg = input; and printf(msg) are mutually exclusive. Only msg = "Access Denied"; and printf(msg); will be executed with the current input.

msg = "Access Denied": am >= untainted

printf(msg): untainted >= am

Solve for all conditions leads to legal flow (am == untainted).

7. Use the same name for a1, a2, b1, and b2; for the wrapper() function, use the name cp and cr for the parameter and return.

wrapper function constraints:

cr >= cp

Call 1 constraints:

a1 >= tainted

cp-1 >= a1

b1 >= cr+1

untainted >= b1

Call 2 constraints:

a2 >= untainted

cp-2 >= a2

b2 >= cr+2

untainted >= b2

Solve for all conditions for call 1 leads to illegal flow (tainted <= untainted).

Solve for all conditions for call 2 leads to legal flow (untainted <= untainted).

8. A good quality initial state of the seed corpus can help the fuzzer reach deeper code paths more quickly by starting from valid or nearly-valid program states. If the seed corpus is of bad quality, it would take a long time for the fuzzer to generate meaningful inputs based on the original input.